MASTER BUILD PROMPT

Stock Market Prediction & Analysis System

ML Pipeline • FastAPI Backend • Angular Services • Database • Deployment

Python 3.11	FastAPI	TensorFlow	Scikit-learn	yfinance	SQLite	Angular 17	Render + Vercel

This is the master build prompt for the full backend and ML system — excluding the UI design (covered separately). Paste this into Google Antigravity to generate the complete Python FastAPI backend, ML training pipeline, Angular services, database schema, and deployment configuration.

Scope: This prompt does NOT include UI components or design — those are in the separate UI Design Prompt PDF. This covers everything needed to make the app actually work: data fetching, model training, REST API, Angular HTTP services, auth, database, and deployment.

What This Prompt Generates

#	Module	Files Generated	Tech
1	Data Pipeline	fetch_data.py, preprocess.py	yfinance, pandas, numpy
2	LSTM Model	lstm_model.py, train_lstm.py	TensorFlow / Keras
3	Random Forest	rf_classifier.py, train_rf.py	Scikit-learn
4	Technical Indicators	indicators.py	pandas-ta / manual calc
5	FastAPI Backend	main.py, routers/, models/	FastAPI, uvicorn
6	Database Schema	database.py, schema.sql	SQLite + SQLAlchemy
7	Auth System	auth.py, jwt_handler.py	python-jose, bcrypt
8	Angular Services	stock.service.ts, auth.service.ts	Angular HttpClient
9	Angular Auth Guard	auth.guard.ts, interceptor.ts	Angular Router
10	Deployment Config	vercel.json, render.yaml, requirements.txt	Vercel + Render

The Master Build Prompt

Copy everything below and paste into Google Antigravity as one single prompt. It is structured in sections so Antigravity can generate each file correctly.

SECTION A — Project Overview & Folder Structure

```
You are building the complete backend, ML pipeline, and Angular service layer for a Stock Market Prediction and Analysis System. The UI already exists — do NOT generate any UI components, HTML templates, or CSS. Generate only the following: Python FastAPI backend, ML model training scripts, Angular TypeScript services and guards, SQLite database schema, and deployment config files.
```

```
PROJECT TECH STACK:
- Backend: Python 3.11, FastAPI, uvicorn
- ML: TensorFlow 2.x / Keras (LSTM), Scikit-learn (Random Forest)
- Data: yfinance library (free Yahoo Finance data)
- Indicators: pandas-ta library
- Database: SQLite (dev) with SQLAlchemy ORM
- Auth: JWT tokens using python-jose, password hashing with bcrypt
- Frontend services: Angular 17 TypeScript (HttpClient only — no components)
- Deployment: Angular on Vercel, FastAPI on Render.com
```

```
FOLDER STRUCTURE TO GENERATE:
backend/
main.py # FastAPI app entry point
database.py # SQLAlchemy engine + session
auth.py # JWT create/verify + bcrypt hashing
routers/
stock.py # /stock endpoints
predict.py # /predict endpoints
market.py # /market/overview endpoint
indicators.py # /indicators endpoints
watchlist.py # /watchlist CRUD endpoints
auth_router.py # /auth/login and /auth/signup
models/
schemas.py # Pydantic request/response models
db_models.py # SQLAlchemy table models
ml/
fetch_data.py # yfinance data fetching
preprocess.py # data cleaning and feature engineering
lstm_model.py # LSTM model architecture
train_lstm.py # LSTM training script
rf_classifier.py # Random Forest model
train_rf.py # RF training script
indicators.py # RSI, MACD, Bollinger, ATR calculation
predict_utils.py # load saved models and run inference
requirements.txt # all Python dependencies
```

```
frontend/src/app/
services/
stock.service.ts # all stock-related HTTP calls
auth.service.ts # login, signup, JWT token management
watchlist.service.ts # watchlist HTTP calls
market.service.ts # market overview HTTP calls
guards/
auth.guard.ts # protect routes from unauthenticated access
interceptors/
auth.interceptor.ts # attach JWT token to every HTTP request
models/
stock.model.ts # TypeScript interfaces for all API responses
```

```
deployment/
vercel.json # Angular deployment config
render.yaml # FastAPI deployment config
```

SECTION B — Data Pipeline (fetch_data.py + preprocess.py)

```
Generate fetch_data.py with these exact requirements:
```

```
- Use yfinance library. Function: fetch_stock_data(ticker: str, period: str = "2y") ->
pd.DataFrame
- ticker format: append .NS for NSE stocks (e.g. "TCS" becomes "TCS.NS")
- Download columns: Open, High, Low, Close, Volume, Adj Close
- Handle errors: if ticker not found, raise HTTPException with clear message
- Add a batch fetch function: fetch_multiple(tickers: list, period="1y") -> dict of
DataFrames
- Cache fetched data in a local /data/ folder as CSV files with timestamp
- If cached CSV is less than 1 hour old, load from cache instead of calling yfinance again
```

```
Generate preprocess.py with these exact requirements:
```

```
- Function: prepare_lstm_data(df: pd.DataFrame, lookback: int = 60) -> tuple(X_train,
y_train, scaler)
- Use MinMaxScaler from sklearn to scale Close prices to range 0-1
- Create sequences: each X sample = last 60 days of scaled Close prices, y = next day
Close price
- Split: 80% train, 20% test. Return X_train, y_train, X_test, y_test, scaler
- Function: prepare_rf_features(df: pd.DataFrame) -> pd.DataFrame
- Add these features: SMA_20, SMA_50, EMA_20, RSI_14, MACD, MACD_signal, volume_ratio
(volume / 20-day avg volume), price_change_pct (daily % change), high_low_range (High -
Low)
- Create target column: signal = 1 (BUY) if next day close > today close by >0.5%, 0
(HOLD) if within 0.5%, -1 (SELL) if drops >0.5%
- Drop NaN rows after feature creation
- Return feature DataFrame ready for Random Forest training
```

SECTION C — ML Models (LSTM + Random Forest)

```
Generate lstm_model.py with these exact requirements:
```

```
- Build using Keras Sequential API (tensorflow.keras)
- Architecture:
Layer 1: LSTM(units=128, return_sequences=True, input_shape=(60, 1))
Layer 2: Dropout(0.2)
Layer 3: LSTM(units=64, return_sequences=False)
Layer 4: Dropout(0.2)
Layer 5: Dense(units=32)
Layer 6: Dense(units=1) # output: next day price (scaled)
- Compile: optimizer=Adam(lr=0.001), loss=mean_squared_error
- Function: build_lstm_model() -> Keras model
- Function: save_model(model, ticker) -> saves to /ml/saved_models/{ticker}_lstm.h5
- Function: load_model(ticker) -> loads from saved path, returns model
```

```
Generate train_lstm.py with these exact requirements:
```

```
- Accept ticker as command line argument: python train_lstm.py --ticker TCS
- Call fetch_stock_data() then prepare_lstm_data()
- Train for 50 epochs, batch_size=32
- Add EarlyStopping callback: monitor=val_loss, patience=10, restore_best_weights=True
- Print training progress and final RMSE on test set
- Save trained model and scaler (save scaler as pickle: {ticker}_scaler.pkl)
- Print: "Model saved. Test RMSE: X.XX"
```

```
Generate rf_classifier.py with these exact requirements:
```

```
- Use RandomForestClassifier from sklearn
- Parameters: n_estimators=200, max_depth=10, random_state=42, class_weight=balanced
- Function: train_rf(ticker) -> trains on prepared features, saves model as
{ticker}_rf.pkl
- Function: predict_signal(ticker, latest_features) -> returns "BUY", "HOLD", or "SELL"
- Function: get_feature_importance(ticker) -> returns dict of feature names and importance
scores
- Print classification report after training (precision, recall, F1 for each class)
- Save model using joblib: joblib.dump(model, path)
```

SECTION D — Technical Indicators (indicators.py)

```
Generate ml/indicators.py with these exact functions. Use pandas-ta library where
possible, fall back to manual calculation if needed:
```

```
- calculate_rsi(df, period=14) -> float: RSI value for latest row (0-100). Also return
"Overbought" if >70, "Oversold" if <30, "Neutral" otherwise.
```

```
- calculate_macd(df) -> dict with keys: macd (float), signal (float), histogram (float),
trend ("Bullish" if macd > signal else "Bearish")
```

```
- calculate_bollinger(df, period=20) -> dict with keys: upper (float), middle (float),
lower (float), current_price (float), position ("Above Upper", "Inside", "Below Lower")
```

```
- calculate_atr(df, period=14) -> dict with keys: atr (float), volatility_level ("High" if atr > 2% of price, "Medium" if 1-2%, "Low" if <1%)
```

```
- calculate_sma(df, periods=[20, 50]) -> dict with SMA values and cross signal ("Golden Cross" if SMA20 > SMA50, "Death Cross" if SMA20 < SMA50)
```

```
- calculate_ema(df, period=20) -> float: latest EMA value
```

```
- get_all_indicators(ticker: str) -> dict: calls all above functions, returns combined dict with keys: rsi, macd, bollinger, atr, sma, ema — ready to send as JSON API response
```

SECTION E — FastAPI Backend (main.py + all routers)

```
Generate main.py:
```

```
- Create FastAPI app with title="StockAI API", version="1.0.0"
- Add CORSMiddleware: allow_origins=["http://localhost:4200", "https://your-vercel-app.vercel.app"], allow_methods=["*"], allow_headers=["*"], allow_credentials=True
- Include all routers with prefixes: /stock, /predict, /market, /indicators, /watchlist, /auth
- Add startup event: create all database tables on app start
- Add root endpoint GET / returning {"message": "StockAI API is running", "version": "1.0.0"}
```

```
Generate routers/stock.py with these endpoints:
- GET /stock/{ticker}?period=1y — fetch OHLCV data, return list of {date, open, high, low, close, volume}
- GET /stock/{ticker}/latest — return latest price, change amount, change percent, last_updated
```

```
Generate routers/predict.py with these endpoints:
- GET /predict/{ticker} — load LSTM model for ticker, run prediction, return {ticker, forecast_7d: [list of 7 prices], forecast_30d: [list of 30 prices], signal: "BUY/HOLD/SELL", confidence: float, last_trained: date}
- POST /predict/train/{ticker} — trigger model training for a ticker (runs train_lstm.py and train_rf.py)
```

```
Generate routers/market.py with these endpoints:
- GET /market/overview — return {sensex: {value, change, change_pct}, nifty50: {value, change, change_pct}, bank_nifty: {value, change, change_pct}, top_gainers: [5 stocks], top_losers: [5 stocks]}
```

```
Generate routers/indicators.py with these endpoints:
- GET /indicators/{ticker} — call get_all_indicators(ticker) and return full JSON
```

```
Generate routers/watchlist.py with these endpoints (all require JWT auth):
- GET /watchlist — return user's saved stocks with latest prices
- POST /watchlist/add — body: {ticker: str} — add to user's watchlist
- DELETE /watchlist/{ticker} — remove from watchlist
```

```
Generate routers/auth_router.py with these endpoints:
- POST /auth/signup — body: {name, email, password} — hash password with bcrypt, save to DB, return JWT
- POST /auth/login — body: {email, password} — verify password, return JWT access token
- GET /auth/me — return current user info from JWT (protected route)
```

SECTION F — Database Schema (SQLite + SQLAlchemy)

```
Generate database.py:
```

```
- Create SQLAlchemy engine: SQLALCHEMY_DATABASE_URL = "sqlite:///./stockai.db"
- Create SessionLocal using sessionmaker
- Create Base = declarative_base()
- Add get_db() dependency function for FastAPI dependency injection
```

```
Generate models/db_models.py with these SQLAlchemy table models:
```

```
Table: users
- id: Integer, primary key, autoincrement
- name: String(100), not null
- email: String(255), unique, not null, indexed
- hashed_password: String(255), not null
- created_at: DateTime, default=now
- is_active: Boolean, default=True
```

```
Table: watchlist
- id: Integer, primary key
- user_id: Integer, ForeignKey(users.id), not null
- ticker: String(20), not null
- added_at: DateTime, default=now
- UniqueConstraint on (user_id, ticker)
```

```
Table: predictions
- id: Integer, primary key
- ticker: String(20), not null
- signal: String(10), not null (BUY/HOLD/SELL)
- forecast_json: Text (store 7-day forecast as JSON string)
- confidence: Float
- created_at: DateTime, default=now
- model_version: String(20)
```

```
Table: price_cache
- id: Integer, primary key
- ticker: String(20), not null
- date: Date, not null
- open: Float, high: Float, low: Float, close: Float, volume: BigInteger
- UniqueConstraint on (ticker, date)
```

```
Generate auth.py:
- SECRET_KEY = "your-secret-key-change-in-production"
- ALGORITHM = "HS256", ACCESS_TOKEN_EXPIRE_MINUTES = 60*24 (24 hours)
- create_access_token(data: dict) -> str (JWT token)
- verify_token(token: str) -> dict (decoded payload, raise 401 if invalid/expired)
- get_password_hash(password: str) -> str (bcrypt hash)
- verify_password(plain: str, hashed: str) -> bool
- get_current_user(token from Authorization header) -> User object
```

SECTION G — Angular Services, Guards & Interceptors

```
Generate frontend/src/app/models/stock.model.ts with these TypeScript interfaces:
```

```
interface OHLCVData { date: string; open: number; high: number; low: number; close:
number; volume: number; }
interface StockLatest { ticker: string; price: number; change: number; change_pct: number;
last_updated: string; }
interface PredictionResult { ticker: string; forecast_7d: number[]; forecast_30d:
number[]; signal: 'BUY' | 'HOLD' | 'SELL'; confidence: number; last_trained: string; }
interface IndicatorResult { rsi: { value: number; status: string }; macd: { macd: number;
signal: number; histogram: number; trend: string }; bollinger: { upper: number; middle:
number; lower: number; position: string }; atr: { atr: number; volatility_level: string };
sma: { sma20: number; sma50: number; cross_signal: string }; }
interface WatchlistItem { ticker: string; price: number; change_pct: number; signal:
string; }
interface AuthResponse { access_token: string; token_type: string; user: { id: number;
name: string; email: string }; }
interface MarketOverview { sensex: { value: number; change: number; change_pct: number };
nifty50: { value: number; change: number; change_pct: number }; top_gainers:
StockLatest[]; top_losers: StockLatest[]; }
```

```
Generate services/stock.service.ts:
- private apiUrl = environment.apiUrl (read from environment.ts)
- getStockData(ticker: string, period = '1y'): Observable -> GET
/stock/{ticker}?period={period}
- getLatestPrice(ticker: string): Observable -> GET /stock/{ticker}/latest
- getPrediction(ticker: string): Observable -> GET /predict/{ticker}
- getIndicators(ticker: string): Observable -> GET /indicators/{ticker}
- getMarketOverview(): Observable -> GET /market/overview
- Use catchError to handle errors and return user-friendly error messages
```

```
Generate services/auth.service.ts:
- login(email: string, password: string): Observable -> POST /auth/login
- signup(name: string, email: string, password: string): Observable -> POST /auth/signup
- logout(): void -> clear token from localStorage
- getToken(): string | null -> return token from localStorage
- isLoggedIn(): boolean -> return true if valid token exists
- getCurrentUser(): Observable -> GET /auth/me
- saveToken(token: string): void -> save to localStorage as 'stockai_token'
```

```
Generate services/watchlist.service.ts:
- getWatchlist(): Observable -> GET /watchlist
- addToWatchlist(ticker: string): Observable -> POST /watchlist/add
- removeFromWatchlist(ticker: string): Observable -> DELETE /watchlist/{ticker}
```

```
Generate guards/auth.guard.ts:
- Implement CanActivate
- Check authService.isLoggedIn()
- If not logged in: redirect to /auth, return false
- If logged in: return true
```

```
Generate interceptors/auth.interceptor.ts:
- Implement HttpInterceptor
- Get token from authService.getToken()
- Clone request and add header: Authorization: Bearer {token}
- Only add header if token exists
- Pass cloned request to next.handle()
```

SECTION H — Environment Config & Requirements

```
Generate backend/requirements.txt with pinned versions:
```

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
sqlalchemy==2.0.30
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.9
yfinance==0.2.40
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.4.2
tensorflow==2.16.1
keras==3.3.3
pandas-ta==0.3.14b
joblib==1.4.2
pydantic==2.7.1
pydantic-settings==2.2.1
python-dotenv==1.0.1
httpx==0.27.0
aiofiles==23.2.1
```

```
Generate frontend/src/environments/environment.ts:
export const environment = {
production: false,
apiUrl: 'http://localhost:8000'
};
```

```
Generate frontend/src/environments/environment.prod.ts:
export const environment = {
production: true,
apiUrl: 'https://your-stockai-backend.onrender.com'
};
```

```
Generate frontend/src/app/app.module.ts additions (do not rewrite full file — only list
what to add):
- Import HttpClientModule from @angular/common/http
- Import HTTP_INTERCEPTORS from @angular/common/http
- Import AuthInterceptor and provide as: { provide: HTTP_INTERCEPTORS, useClass:
AuthInterceptor, multi: true }
```

SECTION I — Deployment Configuration

```
Generate deployment/vercel.json for Angular frontend:
```

```
{
"version": 2,
"builds": [{ "src": "package.json", "use": "@vercel/static-build", "config": { "distDir":
"dist/stock-market-app" } }],
"routes": [{ "src": "/(.*)", "dest": "/index.html" }]
}
```

```
Generate deployment/render.yaml for FastAPI backend:
```

```
services:
- type: web
name: stockai-backend
env: python
buildCommand: pip install -r requirements.txt
startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
envVars:
- key: SECRET_KEY
generateValue: true
- key: PYTHON_VERSION
value: 3.11.0
```

```
Generate backend/.env.example (template for developer to fill):
```

```
SECRET_KEY=change-this-to-a-long-random-string-in-production
DATABASE_URL=sqlite:///./stockai.db
ACCESS_TOKEN_EXPIRE_MINUTES=1440
CORS_ORIGINS=http://localhost:4200,https://your-app.vercel.app
```

```
Generate a startup script backend/setup.py:
- Create /data/ folder if not exists (for CSV cache)
- Create /ml/saved_models/ folder if not exists
- Run: python -c "from database import Base, engine; Base.metadata.create_all(engine)"
- Print: "Database tables created successfully"
- Print: "StockAI backend ready. Run: uvicorn main:app --reload"
```

```
Add a README section at the end explaining:
1. Run python setup.py first to initialize database
2. To train models: python ml/train_lstm.py --ticker TCS then python ml/train_rf.py
--ticker TCS
3. Start backend: uvicorn main:app --reload (runs on port 8000)
4. Start Angular: ng serve (runs on port 4200)
5. For production: push backend to GitHub, connect to Render. Push frontend to GitHub,
connect to Vercel.
```

How to use this prompt in Google Antigravity: Open Antigravity, create a new project, paste the entire content of Sections A through I as one prompt. Antigravity will generate all files in their correct folder structure. After generation, run 'python setup.py' in the backend folder, then train models for at least one ticker (TCS or RELIANCE), then start both servers. The UI Design Prompt PDF covers all Angular components and styling separately.

Quick Reference — Run Order

Step	Command	What it does
1	cd backend && pip install -r requirements.txt	Install all Python dependencies
2	python setup.py	Create DB tables and required folders
3	python ml/train_lstm.py --ticker TCS	Train LSTM for TCS stock
4	python ml/train_rf.py --ticker TCS	Train Random Forest for TCS
5	uvicorn main:app --reload	Start FastAPI on localhost:8000
6	cd frontend && npm install	Install Angular dependencies
7	ng serve	Start Angular on localhost:4200
8	Open http://localhost:4200	View the running app